

Instructivo para desarrollo de la sesión en la Sesión 6 (UNMSM)

Se configura un proyecto en blanco con React-Native expo:

```
npx create-expo-app@latest Ejemplo4 --template blank
```

Se inicia con el siguiente código en la carpeta components dentro del archivo CardWoman.jsx.

También se debe tener dentro de la carpeta data el archivo women.json

```
import { View, Text, StyleSheet, Image } from 'react-native';
import { LinearGradient } from 'expo-linear-gradient';
import React from 'react';
export default function CardWoman({ woman }) {
  return (
    <View style={styles.womanCardContainer}>
      <LinearGradient
        colors={['rgba(185,28,89,.8)', 'rgba(194,149,214,.7)', 'rgba(147,123,191,.6)']}
        style={styles.background}
      />
      <Text style={styles.titleText}>{woman.name} {woman.lastName}</Text>
      <Image source={{ uri: woman.photo }} style={{ width: 200, height: 200, borderRadius: 10 }} />
      <Text>{woman.nationality}</Text>
      <Text style={styles.contentText}>{woman.bio}</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  womanCardContainer: {
    flex: 1,
    paddingHorizontal: 20,
    justifyContent: 'center',
    alignItems: 'center',
    height: 500,
  },
  titleText: {
    fontSize: 24,
    marginBottom: 20,
  },
  contentText: {
    marginTop: 10
  },
  background: {
    position: 'absolute',
    left: 0,
    right: 0,
    top: 0,
    height: 490,
    borderRadius: 20,
  },
});
```

También el siguiente código dentro de APP.jsx

```
import { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, ScrollView, Image, FlatList, Button } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import women from './data/women.json';
import CardWoman from './components/CardWoman';
export default function App() {
  const [count, setCount]=useState(0);
  const handlePress= () => {
    setCount(count+1);
  };
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <View style={styles.container}>
```

```

<Text>Desarrollo de Aplicaciones Móviles</Text>
<Text style={{fontSize: 40, textAlign: 'center'}}>{count}</Text>
<Button onPress={handlePress} title="add one"/>

<FlatList
  data={women}
  keyExtractor={item => item.id}
  renderItem={({item}) =>
    <CardWoman woman={item}/>
  }
/>
<StatusBar style="auto" />
</View>
</SafeAreaView>
);
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  imageSize: {
    width: 100,
    height: 100,
  }
});

```

Instalar las dependencias necesarias.

```
npx expo install expo-linear-gradient
```

```
npx expo install react-native-safe-area-context
```

Ejecutar la aplicación con el comando `npx expo start` o `npm start -- --reset-cache`

A continuación, se borra las siguientes líneas del código en APP.js. Se va mostrar solo 5 mujeres, para simular la consulta paginada a una API y cargar las siguientes páginas con pull to Refresh

```

/*const [count, setCount]=useState(0);
const handlePress= () => {
  setCount(count+1);
},*/

```

```

{/*<Text style={{fontSize: 40, textAlign: 'center'}}>{count}</Text>
  <Button onPress={handlePress} title="add one"/>*/}

```

Y se agrega la siguiente línea de código, antes de return, dentro de APP:

```
const [data, setData] = useState(women.slice(0,5));
```

Y también dentro de la FlatList se cambia `data={women}` por:

```
data={data}
```

Con ello solo se debe mostrar 5 registros del archivo JSON (es decir 5 mujeres) (teclea r de ser necesario)

A continuación, queremos refrescar la pantalla para cargar las siguientes mujeres (pull to refresh), por ello se debe importar el componente de react native llamado RefreshControl

```
import { StyleSheet, Text, View, ScrollView, Image, FlatList, Button, RefreshControl } from 'react-native';
```

Borrar los componentes que no se usan(opcional). Luego dentro de Flatlist (antes del render), se habilita la prop que detecta el gesto de seguir cargando imágenes, el cual ejecutará la función handleRefresh cuando se dé el gesto de refrescar

```
refreshControl={
  <RefreshControl onRefresh={handleRefresh} />
}
```

La función handleRefresh tiene el objetivo de jalar los siguientes 5 registros del archivo JSON, para ello debe cambiar el estado del componente para renderizar la UI nuevamente

Se escribe la función handleRefresh inicialmente con el siguiente contenido (después del slice):

```
const handleRefresh = () => {
  console.log("refrescando...");
};
```

Se comprueba que la aplicación intenta algo infinitamente (sale la imagen de carga y no se sale, además de mostrar el texto refrescando... en la consola). Como mejora se debe colocar un activity Indicator.

A continuación, se utiliza la prop refreshing del componente RefreshControl que sirve para decirle al componente (y de paso a la FlatList) que se está en medio de un proceso (con true o false)

```
refreshControl={
  <RefreshControl refreshing={refreshing} onRefresh={handleRefresh} />
}
```

Ahora se debe usar useState para controlar el estado de la variable refreshing. Se coloca el siguiente código después del slice

```
const [refreshing, setRefreshing] = useState(false);
```

Con esto la variable refreshing se inicializa en false, para decirle al componente que no haga nada aún con el refresh hasta que esté en true. La idea es que con la función handlePress se colocará en true.

Para ello se coloca el siguiente código (se borra el console.log("refrescando...")):

```
const handleRefresh = () => {
  setRefreshing(true);
};
```

Se necesita cargar los siguientes 5 registros del archivo JSON, para ello la función handlePress se completa con el siguiente código

```
const handleRefresh = () => {
  setRefreshing(true);
  setTimeout ( () => {
    const newLoadedCount = loadedCount + 5;
    const nextItems = women.slice(loadedCount, newLoadedCount);
    setData((prev) => [...prev, ...nextItems]);
    setLoadedCount(newLoadedCount);
    setRefreshing(false);
  }, 1500);
};
```

Se debe inicializar y controlar el valor de la variable loadedCount con un useState, para ello se coloca la siguiente instrucción antes del slice

```
const[loadedCount, setLoadedCount] = useState(5);
```

Adicionalmente se corrige la instrucción: `const [data, setData] = useState(women.slice(0,5))` por lo siguiente:

```
const [data, setData] = useState(women.slice(0,loadedCount));
```

Adicionalmente la línea del flatList que contiene keyExtractor, se cambia por el siguiente código

```
keyExtractor={(item)=> item.id.toString()}}
```

item.id: Se asume que cada objeto en tu conjunto de datos (item) tiene una propiedad id que es única para ese elemento, algo común en APIs y bases de datos.

.toString(): A menudo, las claves deben ser cadenas de texto. Al usar toString(), te aseguras de que el ID, que podría ser un número, se convierta a un formato de cadena que keyExtractor espera, cumpliendo con los requisitos de las claves de React.

Siempre que hay problemas en la ejecución de la aplicación se puede utilizar el comando para limpiar la caché.

```
npm start -- --reset-cache
```

Ahora se va utilizar ListHeaderComponent y ListFooterComponent dentro de FlatList, que hace lo siguiente:

- En el encabezado (header) se indica que se debe hacer pull para cargar registros, se muestra la cantidad total de registros del JSON y dice cuando se acabaron.
- En el pie (footer) se muestra cuántos registros se han cargado, cuántos faltan y un mensaje de "terminó" cuando se alcanza el final.

Se adiciona dentro del Flatlist antes del render:

```
ListHeaderComponent={listHeader}
```

```
ListFooterComponent={listFooter}
```

Las funciones listHeader, listFooter deben contener el siguiente código, se colocan antes del return, después del handleRefresh:

```
// Header que muestra instrucciones y total de registros
const listHeader = () => {
  if (loadedCount >= totalRecords) {
    return (
      <Text style={styles.headerFooterText}>
        Se acabaron los registros. Total: {totalRecords}
      </Text>
    );
  } else {
    return (
      <Text style={styles.headerFooterText}>
        Haga pull para cargar registros. Total registros disponibles: {totalRecords}
      </Text>
    );
  }
};

// Footer que muestra estado de carga
```

```
const listFooter = () => {
  if (loadedCount >= totalRecords) {
    return <Text style={styles.headerFooterText}>Carga terminada. Total registros cargados: {loadedCount}</Text>;
  } else {
    const remaining = totalRecords - loadedCount;
    return (
      <Text style={styles.headerFooterText}>
        Cargados: {loadedCount}. Faltan: {remaining} registros por cargar.
      </Text>
    );
  }
};
```

Para completar el código se agrega la siguiente instrucción, después del export default :

```
const totalRecords = women.length;
```

La función handleRefresh debe tener el siguiente contenido:

```
const handleRefresh = () => {
  setRefreshing(true);
  setTimeout(() => {
    const newLoadedCount = Math.min(loadedCount + 5, totalRecords);
    const nextItems = women.slice(loadedCount, newLoadedCount);
    setData((prev) => [...prev, ...nextItems]);
    setLoadedCount(newLoadedCount);
    setRefreshing(false);
  }, 1500);
};
```

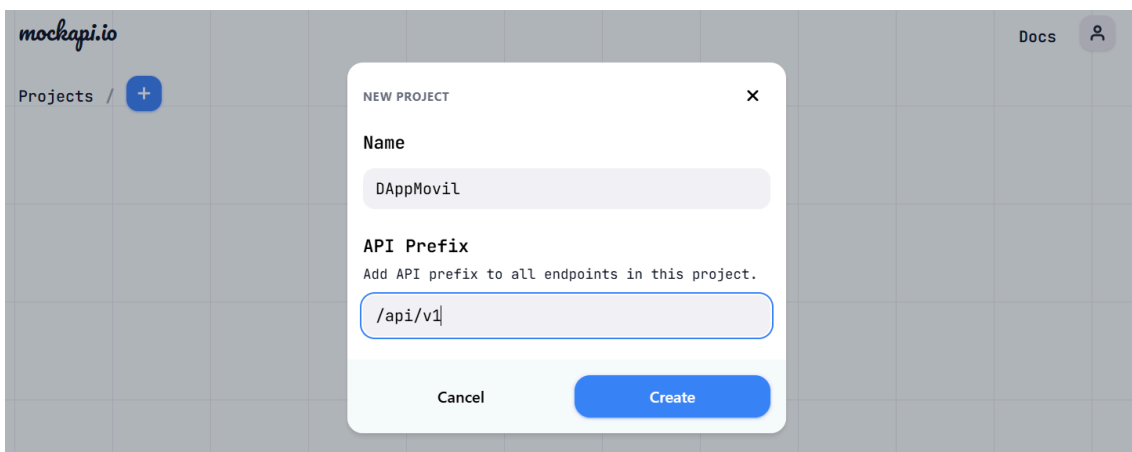
También se agrega el siguiente estilo dentro de la hoja de estilos antes de imageSize

```
headerFooterText: {
  padding: 10,
  fontSize: 14,
  textAlign: 'center',
  fontWeight: 'bold',
},
```

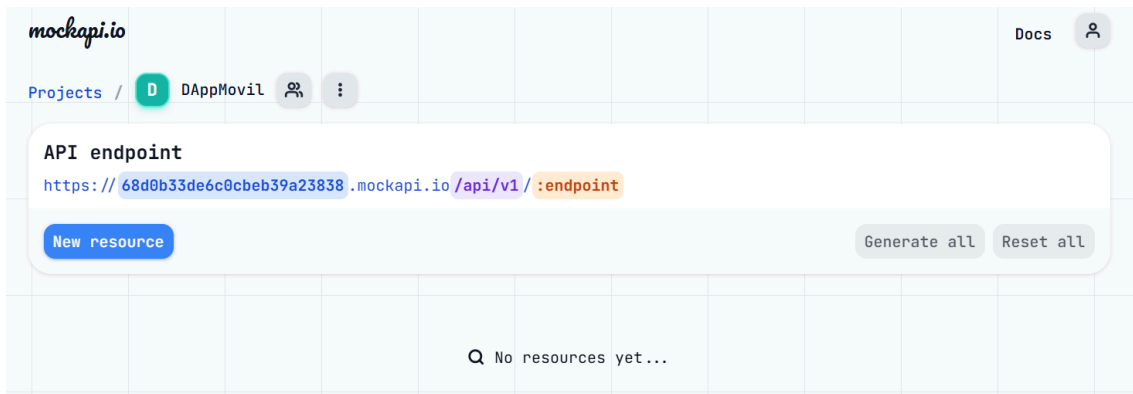
A continuación, nos llevamos los registros del archivo women.json en algún servicio de APIs, por ejemplo mock API

Visitar la url <https://mockapi.io/> y registrarse, puede ser con Gmail

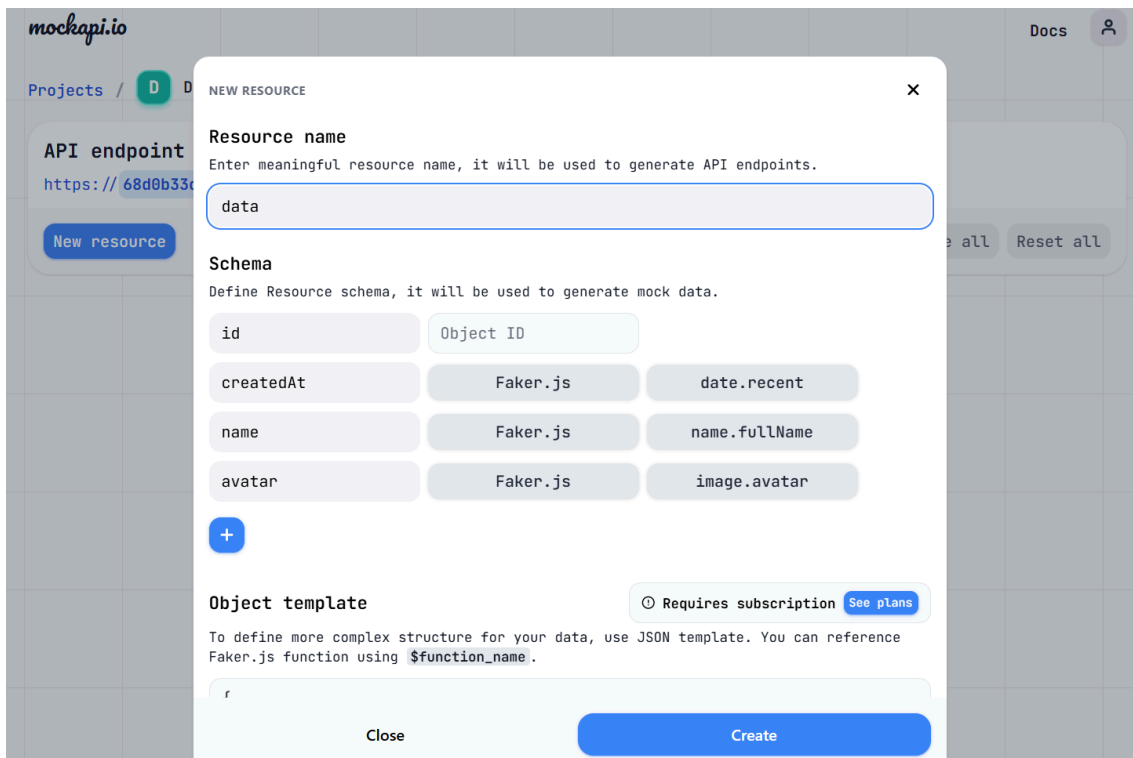
Se crea un nuevo proyecto (click al signo +), se coloca un nombre y también el API Prefix, puede ser /api/v1



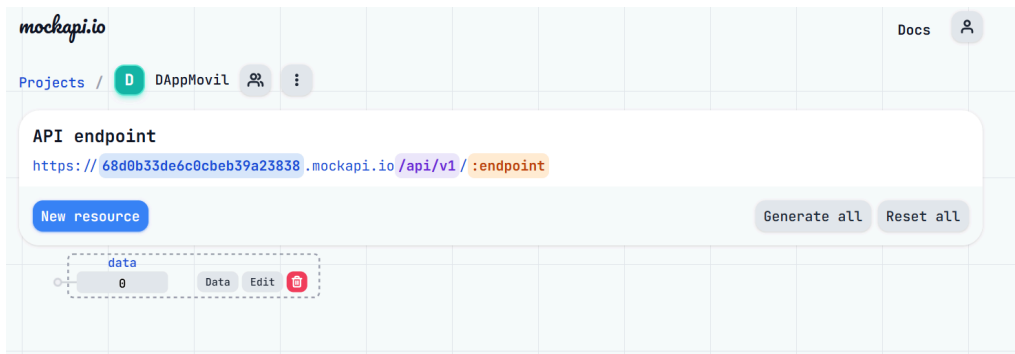
Dentro del proyecto se crea un nuevo recurso, haciendo click al botón New resource:



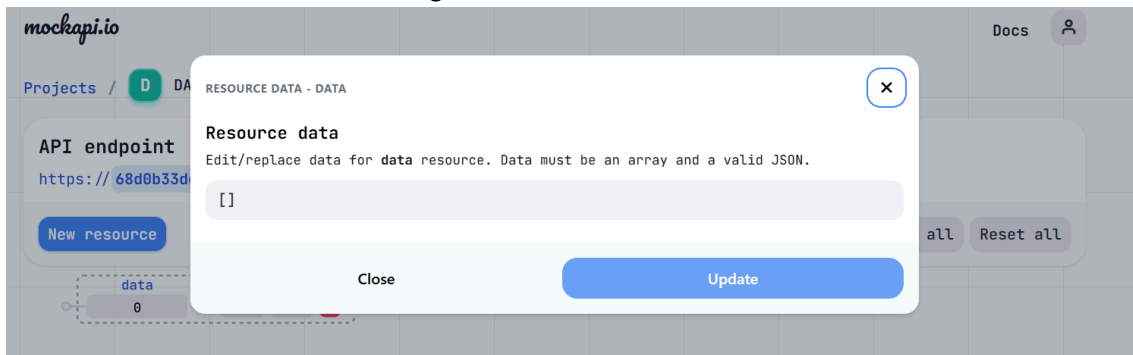
Se le coloca el nombre de recurso, en nuestro caso data y se hace click en el botón create



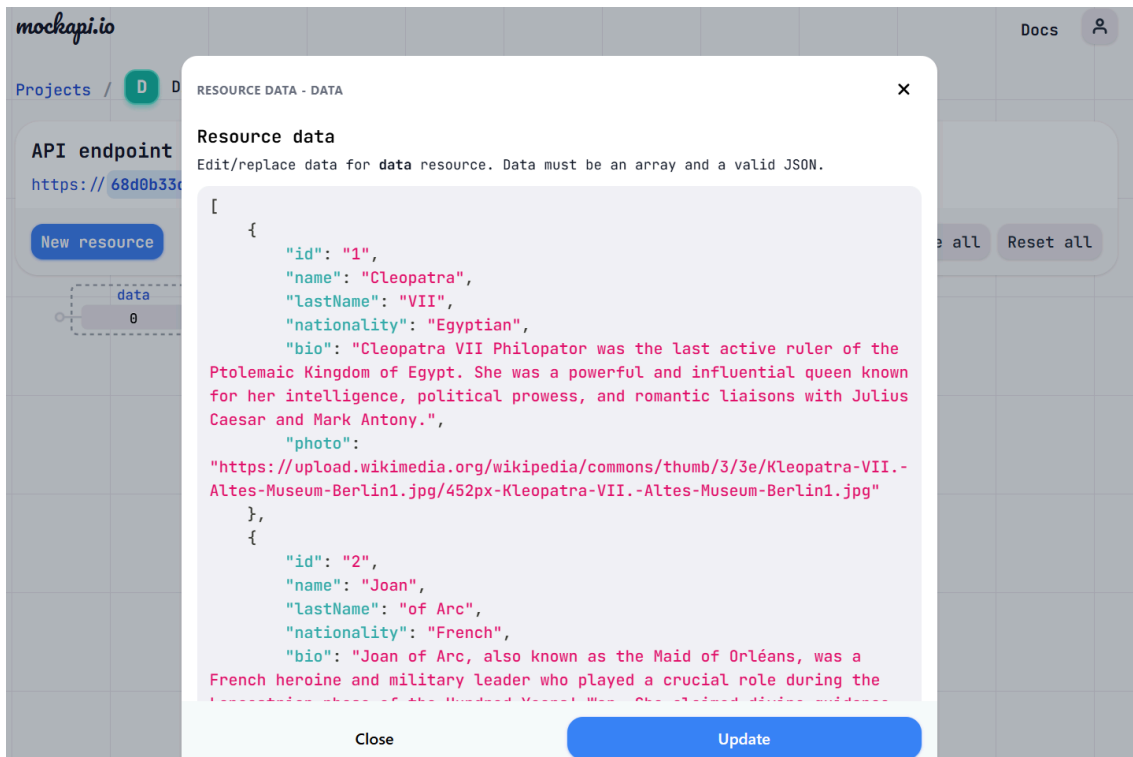
Ahora se debe colocar el contenido del archivo json que hemos estado usando, se hace click en el botón gris que tiene la palabra Data:



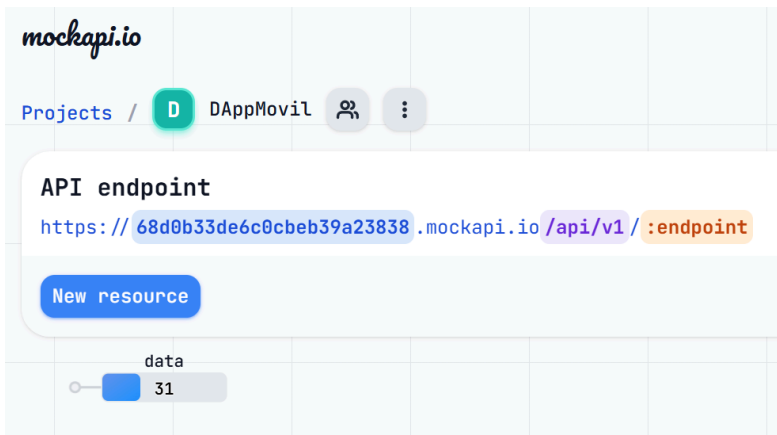
Inicialmente el contenido es un arreglo vacío.



En ese campo se debe colocar el contenido del archivo JSON que estamos trabajando (women.json) y luego al botón Update



Se copia el URL respectivo del recurso creado:



<https://68d0b33de6c0cbeb39a23838.mockapi.io/api/v1/data>

Para procesar esta API, se comenta todo el proceso de refreshing, porque esta API no está paginada. Se inicia por comentar `ListHeaderComponent={listHeader}`, `ListFooterComponent={listFooter}` y las funciones respectivas.

```
// ListHeaderComponent={listHeader}
// ListFooterComponent={listFooter}
```

```
/*
// Header que muestra instrucciones y total de registros
const listHeader = () => {
  if (loadedCount >= totalRecords) {
    return (
      <Text style={styles.headerFooterText}>
        Se acabaron los registros. Total: {totalRecords}
      </Text>
    );
  } else {
    return (
      <Text style={styles.headerFooterText}>
        Haga pull para cargar registros. Total registros disponibles: {totalRecords}
      </Text>
    );
  }
};

// Footer que muestra estado de carga
const listFooter = () => {
  if (loadedCount >= totalRecords) {
    return <Text style={styles.headerFooterText}>Carga terminada. Total registros cargados: {loadedCount}</Text>;
  } else {
    const remaining = totalRecords - loadedCount;
    return (
      <Text style={styles.headerFooterText}>
        Cargados: {loadedCount}. Faltan: {remaining} registros por cargar.
      </Text>
    );
  }
};
*/
```

Luego se comenta (o se borra) todo el `handleRefresh`

```
/*
const handleRefresh = () => {
  //console.log("refrescando...");
  setRefreshing(true);
  setTimeout(() => {
    //const newLoadedCount = loadedCount + 5;
    const newLoadedCount = Math.min(loadedCount + 5, totalRecords);
    const nextItems = women.slice(loadedCount, newLoadedCount);
    setData((prev) => [...prev, ...nextItems]);
    setLoadedCount(newLoadedCount);
    setRefreshing(false);
  }, 1500);
};
*/
```

Lo mismo con `const[refreshing, setRefreshing] =useState(false)`; (se borra o se comenta)

```
//const[refreshing, setRefreshing] =useState(false);
```


También se comenta (o se borra) la prop refreshControl del FlatList

```
// refreshControl={<RefreshControl refreshing={refreshing} onRefresh={handleRefresh} />}
```

Se comentan la siguientes líneas, donde figura el siguiente useState

```
//const totalRecords = women.length;  
//const[loadedCount, setLoadedCount] = useState(5);
```

Asimismo, se comenta el import del archivo women.json

```
//import women from './data/women.json';
```

Sólo se deja el siguiente useState con el contenido mostrado (se inicializa el estado con un arreglo vacío, es decir data es un arreglo vacío):

```
const [data, setData] = useState([]);
```

A continuación, se explica las siguientes instrucciones que van a permitir obtener la información de la API (se coloca antes del default function, verificar que pasa si se pone dentro del componente):

```
const URL = "https://68d0b33de6c0cbeb39a23838.mockapi.io/api/v1/data";  
const getData = async (url) => {  
  const res = await fetch(url);  
  const women = await res.json();  
  console.log(women);  
};
```

Con fines de prueba modificamos el useState de la siguiente manera:

```
const [data, setData] = useState(getData(URL));
```

Se comprueba que lo obtenido de la API se imprime en la consola (por la sentencia console-log(women)), pero en la APP no se refleja nada

Ahora necesitamos llevar esa información de la API a la interfaz de usuario de la aplicación móvil, debieramos usar la función setData creada con el hook useState.

Podríamos hacerlo con una interacción del usuario, es decir el usuario presiona un botón, luego llamamos a una función que actualiza el estado con setData y cargamos la info. Pero no queremos hacer eso, queremos que se cargue sin la interacción con el usuario.

Antes, para entender mejor lo que estamos diciendo, vamos a hacer un condicional para definir lo que se vá mostrar en la interfaz de usuario cubrimos el FlatList con el siguiente código:

```
{data.length ?  
<FlatList  
  data={data}  
  keyExtractor={item=> item.id}  
  
  renderItem={({item}) =>  
    <CardWoman woman={item}/>  
  }  
/>  
:  
<ActivityIndicator size={'large'} />
```

Pero antes se debe incluir importar el componente ActivityIndicator

```
import { StyleSheet, Text, View, ScrollView, Image, FlatList, Button, RefreshControl, ActivityIndicator } from 'react-native';
```

Con ello se demuestra que data es un arreglo vacío (valor nulo o cero), ya que se muestra el ActivityIndicator. Entonces ¿Qué pasó con getData(URL) en la siguiente sentencia?

```
const [data, setData] = useState(getData(URL));
```

La conclusión es que con useState, data debe ser inicializado con un valor constante y luego sólo modificado por setData, para que la interfaz de usuario se actualice.

Se vuelve al arreglo vacío:

```
const [data, setData] = useState([]);
```

Debemos esperar que se cargue el componente y luego cargar la info, es un efecto secundario que debemos hacerlo con useEffect (se explica el tema de funciones puras e impuras, una función que hace fetch se vuelve impura)

En primer lugar se importa useEffect

```
import { useEffect, useState } from 'react'
```

Luego dentro del componente principal, después del useState, se coloca el siguiente código:

```
useEffect(() => {  
  const getData = async (url) => {  
    const res = await fetch(url);  
    const women = await res.json();  
    setData(women);  
  };  
  getData(URL);  
});
```

Ahora ya se puede comentar el siguiente código:

```
const getData = async (url) => {  
  const res = await fetch(url);  
  const women = await res.json();  
  console.log(women);  
};
```

Cuando una app móvil carga datos de una API, puede darse distintos escenarios de problemas o fallas, se debe gestionar este tema. Típicamente puede existir un error que no me deja cargar los datos, esta en proceso de carga, u todo está OK. La renderización condicional que hicimos ayuda con en este tema, pero vamos a usar un patrón típico de gestión de errores

Agregar el siguiente código, después del useState([]):

```
const [error, setError] = useState(null);  
const [isLoading, setIsLoading] = useState(true);
```

El useEffect debe ahora tener el siguiente contenido (reemplazar por el existente)

```
useEffect(() => {  
  const getData = async (url) => {  
    setIsLoading(true);  
    try {  
      const res = await fetch(url);  
      const women = await res.json();  
      setData(women);  
    } catch (err) {
```

```

        setError(err.message);
    } finally {
        setIsLoading(false);
    }

    };
    getData(URL);
  }, []);

```

Luego cambiar el siguiente bloque de código

```

{data.length ?
  <FlatList
    data={data}
    keyExtractor={item => item.id.toString()}
    // refreshControl={<RefreshControl refreshing={refreshing} onRefresh={handleRefresh} />}
    // ListHeaderComponent={listHeader}
    // ListFooterComponent={listFooter}

    renderItem={({item}) =>
      <CardWoman woman={item}/>
    }
  />
:
  <ActivityIndicator size='large'/>
}

```

Por el siguiente:

```

{isLoading ?
  <ActivityIndicator size="large" />
:
  error ?
    <Text>{error}</Text>
  :
    <FlatList
      data={data}
      keyExtractor={item => item.id.toString()}
      renderItem={({ item }) =>
        <CardWoman woman={item} />
      }
    />
}

```

El código final debe ser similar al siguiente:

```

import { useState,useEffect } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, FlatList, ActivityIndicator } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import CardWoman from './components/CardWoman';

const URL = "https://68d0b33de6c0cbeb39a23838.mockapi.io/api/v1/data";

export default function App() {
  const [data, setData] = useState([]);
  const [error, setError] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const getData = async (url) => {
      setIsLoading(true);
      try {
        const res = await fetch(url);
        const women = await res.json();
        setData(women);
      } catch (err) {

```

```

        setError(err.message);
    } finally {
        setIsLoading(false);
    }

    };
    getData(URL);
  }, []);
  return (
    <SafeAreaView style={{ flex: 1 }}>
    <View style={styles.container}>
      <Text>Desarrollo de Aplicaciones Móviles</Text>
      {isLoading ?
        <ActivityIndicator size="large" />
        :
        error ?
          <Text>{error}</Text>
          :
          <FlatList
            data={data}
            keyExtractor={item => item.id.toString()}
            renderItem={({ item }) =>
              <CardWoman woman={item} />
            }
          />
        }
      <StatusBar style="auto" />
    </View>
  </SafeAreaView>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerFooterText: {
    padding: 10,
    fontSize: 14,
    textAlign: 'center',
    fontWeight: 'bold',
  },
  imageSize: {
    width: 100,
    height: 100,
  }
});

```

Se puede optimizar la gestión de errores con el siguiente código:

```

import { useState, useEffect } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, FlatList, ActivityIndicator } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import CardWoman from './components/CardWoman';

const URL = "https://68d0b33de6c0cbeb39a23838.mockapi.io/api/v1/data";

export default function App() {
  const [data, setData] = useState([]);
  const [error, setError] = useState(null);
  const [isLoading, setIsLoading] = useState(false);

  // Función para obtener datos, manejando también errores por status HTTP distinto de 200-299
  const fetchData = async () => {
    setIsLoading(true);
    setError(null);
    try {

```

```

const response = await fetch(URL);
if (!response.ok) {
  // Generamos error si la respuesta no es OK
  throw new Error(`Error HTTP: ${response.status}`);
}
const json = await response.json();
setData(json);
} catch (err) {
  setError(err.message || 'Error desconocido');
} finally {
  setIsLoading(false);
}
};

useEffect(() => {
  fetchData();
}, []);

return (
  <SafeAreaView style={{ flex: 1 }}>
    <View style={styles.container}>
      <Text>Desarrollo de Aplicaciones Móviles</Text>
      {isLoading ? (
        <ActivityIndicator size="large" />
      ) : error ? (
        <Text style={{ color: 'red', padding: 10 }}>{error}</Text>
      ) : (
        <FlatList
          data={data}
          keyExtractor={({item}) => item.id.toString()}
          renderItem={({item}) => <CardWoman woman={item} />}
          contentContainerStyle={{ paddingBottom: 20 }}
        />
      )}
      <StatusBar style="auto" />
    </View>
  </SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerFooterText: {
    padding: 10,
    fontSize: 14,
    textAlign: 'center',
    fontWeight: 'bold',
  },
  imageSize: {
    width: 100,
    height: 100,
  },
});

```

Cambios y optimizaciones realizadas :

- Se inicializa isLoading en false y solo se activa dentro del fetch para reflejar adecuadamente el estado de carga.
- Se resetea el valor de error a null cada vez que se inicia la petición para evitar mostrar mensajes previos incorrectamente.
- Se añade revisión de response.ok para detectar errores HTTP explícitamente, no solo errores de red.

- Se muestra un mensaje de error con estilo rojo para que el usuario lo identifique fácilmente.
- Se externaliza la función `fetchData` para claridad y posible reutilización.
- Se añade `contentContainerStyle` al `FlatList` para mejorar estética en scroll.
- Con estos cambios, el código queda más robusto y claro para gestionar errores comunes en peticiones `fetch` dentro de React Native para Expo.